CODESPECT

Sub-Account Strategy Framework

SECURITY ASSESSMENT REPORT

July 2, 2026

Prepared for:

Hyperwave

Contents

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty programme to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for Hyperwave Prime. The review targeted the changes introduced in pull request #86, which add a **sub-account strategy framework** on top of Hyperwave's `BoringVault`-based vault. The framework lets the vault allocate its base asset to authorised strategy sub-accounts under per-strategy limits, and lets each strategist execute a pre-approved set of calls on external protocols through a Merkle-verified manager.

The scope comprised four Solidity contracts which were added in PR-86: two role contracts (`SubAccountFundManager` and `SubAccountWithMerkleVerification`) and two decoder-and-sanitizer contracts (`YieldBasisDecoderAndSanitizer` for the Yield Basis leveraged-token strategy in the aprimeUSD product, and `WrappedNativeDecoderAndSanitizer` for wrapping and unwrapping the native asset). All reviewed files are summarised in subsection 5.1.

The audit was performed using:

a) Manual analysis of the codebase.

b) Dynamic analysis of the smart contracts and execution testing.

Organisation of the document is as follows:

- **Section 4** presents this executive summary and the issue counts.
- **Section 5** summarises the audit.
- **Section 6** describes the functionality and architecture of the in-scope contracts.
- **Section 7** presents the issues.
- **Section 8** presents the threat model.
- **Section 9** discusses the documentation provided by the client for this audit.
- **Section 10** presents the compilation and tests.

CODESPECT found fourteen points of attention, three classified as `Low` and eleven classified as `Informational`. All of the issues are summarised in the table below.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	3	0
Informational	0	7	4
Total	**0**	**10**	**4**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	Sub-Account Strategy Framework
Type of Project	Vault Strategy Framework (EVM Smart Contracts)
Duration of Engagement	2 Days
Duration of Fix Review Phase	1 Day
Draft Report	July 2, 2026
Final Report	July 2, 2026
Repository	boring-vault
Commit (Audit)	6f6cd157b9aa4f63263e70339bbc37106cede493
Commit (Final)	bcfacae4894eebabcaee2bee076d0336418ea07b
Documentation Assessment	High
Test Suite Assessment	High
Auditors	cholakovvv, 0xSynthrax

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	Roles/SubAccountWithMerkleVerification.sol	213
2	Roles/SubAccountFundManager.sol	167
3	DecodersAndSanitizers/hyperwave/aprimeusd/YieldBasisDecoderAndSanitizer.sol	212
4	DecodersAndSanitizers/hyperwave/common/WrappedNativeDecoderAndSanitizer.sol	22
	Total	**614**

5.2 Findings Overview

	Finding	Severity	Update
1	Repayment path reverts if a blocklist-capable baseAsset blocks the strategy or the vault	Low	Fixed
2	`transferFundFromVault(...)` uses unchecked `transfer(...)` call	Low	Fixed
3	`withdraw(...)` slippage floor misprices staker shares when `unstake` is true	Low	Fixed
4	`transferFundBackToVault(...)` floors `allocated` to zero while transferring the full amount	Info	Acknowledged
5	Slippage floors collapse to zero on `deposit(...)` and `withdraw(...)`	Info	Fixed
6	`transferFundFromVault(...)` reverts for a registered strategy whose limit is unset or non-numeric	Info	Acknowledged
7	Merkle leaf binds decoder and target addresses but not their code	Info	Acknowledged
8	Authorized wrap/unwrap leaves cannot execute against the non-payable sub-account	Info	Fixed
9	Merkle authorization assumes the decoder and target share the same function for a given selector	Info	Acknowledged
10	`maxSlippage` and `slippageBase` are global, not configurable per `poolId`	Info	Fixed
11	`withdraw(...)` reuses the deposit pool allowlist, so disabling a pool freezes existing positions	Info	Fixed
12	Interleaved verify/execute makes the YieldBasis slippage floor depend on a live preview	Info	Fixed
13	Direct `manage(...)` call will execute successfully even when the contract is paused	Info	Fixed
14	Missing zero-address validation in `SubAccountFundManager` constructor	Info	Fixed

6 System Overview

Pull request #86 introduces a **sub-account strategy framework** on top of Hyperwave's `BoringVault`-based vault. The framework lets the vault put its base asset to work in external strategies without handing a strategist unbounded control: a strategist may only make calls that are pre-committed in a per-strategist Merkle tree, and a strategy sub-account may only pull vault funds up to a per-strategy allocation limit. Four contracts were in scope: the Merkle-gated call router (`SubAccountWithMerkleVerification`), the base-asset allocation manager (`SubAccountFundManager`), and two decoder-and-sanitizer contracts that describe and constrain individual strategy calls (`YieldBasisDecoderAndSanitizer` and `WrappedNativeDecoderAndSanitizer`).

The vault itself (`BoringVault`) is the custodian: it holds the funds and executes arbitrary calls through its `manage(target, data, value)` hook. It is out of scope and treated as a trusted primitive, as are the `Configuration` store, the `Auth`/`Authority` access-control layer, the `BaseDecoderAndSanitizer` base contract, and the external protocols the strategies interact with.

Architecture: Router, Fund Manager, and Decoders

The framework layers two `Auth`-gated contracts on the vault, each constructed with `Authority(address(0))` so authorisation initially resolves to the owner and later to whatever `Authority` the owner installs. `SubAccountWithMerkleVerification` is the router that decides which calls a strategist may make; `SubAccountFundManager` is the accountant that moves the vault's base asset in and out of strategy sub-accounts under a limit. Decoder-and-sanitiser contracts sit beneath the router as per-call validators: for each proposed call, the router hands the raw calldata to a decoder, which both extracts the address arguments that the call is allowed to touch and, where it chooses to, rejects calls that fall outside a policy the static Merkle tree cannot express.

The two paths ultimately rest on the same execution primitive. The router calls `functionCallWithValue` on a target directly; the fund manager calls `boringVault.manage` to move vault-held funds. A strategist's calls therefore reach external protocols through the router, and reach the vault's balance through the fund manager, which is itself one of the targets a strategist can be permitted to call.

SubAccountWithMerkleVerification

Each strategist address maps to its own Merkle root in `mapping(address => bytes32) manageRoot`. This is the permission set: a strategist can execute a call only if the call's shape is a leaf in *their* tree. Each leaf is

```
keccak256(abi.encodePacked(decoderAndSanitizer, target, valueIsNonZero, selector, packedArgumentAddresses))
```

so the tree pins, per allowed call: which decoder must validate it, which target it goes to, whether the ETH value is non-zero (a boolean, not the exact amount), the 4-byte function selector, and the exact set and order of address arguments the decoder surfaces. Non-address arguments (amounts, pool ids, booleans) are not bound by the leaf; they are constrained only if the decoder validates them.

The strategist entry point, `manageWithMerkleVerification`, takes five index-aligned `calldata` arrays (`manageProofs`, `decodersAndSanitizers`, `targets`, `targetData`, `values`) and, after checking `isPaused` and that every array matches `targets.length`, loads the caller's own root and processes each call in turn:

a. **Decode and sanitize.** The full `targetData` is forwarded to the leaf's decoder via `staticCall`; the decoder returns the packed address arguments. Because it is a static call the decoder cannot change state, and because a revert propagates, a decoder that rejects the call (disallowed pool, a min-out below the slippage floor, an unimplemented selector) aborts the whole batch.

b. **Verify.** The leaf is rebuilt from the decoder's output, the call's selector (`bytes4(targetData)`), the target, and the `value > 0` flag, and checked against the caller's root with `MerkleProofLib.verify`. A failure reverts.

c. **Execute.** The verified call is executed via the internal `manage`, which performs `target.functionCallWithValue(data, value)`.

Verification and execution are interleaved per call rather than verified-all-then-executed. Two administrative controls round out the contract: `setManageRoot(strategist, root)` (the sole means of granting or revoking a strategist's permissions) and `pause`/`unpause`, both `requiresAuth`. This commit also adds a standalone `public manage(target, data, value)` for emergency use; it is the same function the batch path calls internally, but reached directly it is gated by `requiresAuth` only, with no Merkle proof, no decoder staticcall, and no pause check. It is intended for the trusted owner/authority, not for strategists.

Decoders and Sanitizers

A decoder-and-sanitizer is a contract, named inside each leaf, that the router `staticCall`s with the target's calldata. It mirrors the target's function selectors and returns `abi.encoded` bytes of the address arguments that must be committed in the leaf. It has two jobs: **extraction** (surface the addresses the tree binds) and **sanitization** (revert on input the tree cannot police). Two examples were in scope.

`WrappedNativeDecoderAndSanitizer` is the trivial case: extending `BaseDecoderAndSanitizer`, it mirrors WETH-style `deposit()` and `withdraw(uint256)` and returns empty bytes for both, since neither wrap nor unwrap carries an address argument. Its only constraint is that the Merkle leaf selects these selectors against the correct wrapped-native target.

`YieldBasisDecoderAndSanitizer`, the decoder for the Yield Basis leveraged-token strategy in the aprimeUSD product, is deliberately richer and, unusually for a decoder, **stateful and governance-controlled**. It inherits OpenZeppelin `AccessControl` with a single self-administered `GOVERNANCE_ROLE`. Governance maintains a pool allowlist (`isAllowedPoolId`), a per-pool leveraged-token address (`poolIdToLT`) used for previews, and a slippage configuration (`maxSlippage`, `slippageBase`) that is validated on every write to keep `maxSlippage < slippageBase` and `slippageBase` non-zero. Its decoder functions:

- `approve(spender, amount)` returns `abi.encodePacked(spender)`, binding the approved spender to the leaf.
- `deposit(...)` requires the pool to be allowlisted and its leveraged token set, then computes `expectedShares` via `ILT.preview_deposit` and enforces that the strategist's `minShares` is at least `expectedShares · (slippageBase − maxSlippage) / slippageBase`, reverting `MinSharesTooLow` otherwise. It commits no address.
- `withdraw(...)` is symmetric, using `ILT.preview_withdraw` to bound `minAssets`, and returns `abi.encodePacked(receiver)` to bind the withdrawal recipient.
- `deposit_crvusd`, `redeem_crvusd`, `transferFundFromVault`, and `transferFundBackToVault` are no-op decoders that commit nothing, and a `fallback` reverts `FunctionNotImplemented` for any selector without an explicit decoder.

So this decoder does two things at once: it extracts addresses like any decoder, and it enforces an on-chain slippage floor on every proven deposit and withdraw, layered on top of the Merkle allow-list. The presence of `transferFundFromVault`/`transferFundBackToVault` decoder stubs is the hook that lets a strategist route calls whose target is the `SubAccountFundManager` through the router.

SubAccountFundManager

This contract moves the vault's single `baseAsset` between the `boringVault` and the strategy sub-accounts, holding three immutables (the vault, the base asset, and the `configuration` store) and two mappings: `strategyLimitKey` (strategy address to the `Configuration` key naming its limit) and `allocated` (per-strategy outstanding allocation). A strategy becomes a recognised sub-account simply by an admin setting a non-empty `strategyLimitKey` for it via `setStrategyLimitKey`; setting an empty key removes recognition.

- `transferFundFromVault(amount)` lets a registered strategy draw `amount` of base asset out of the vault into its own sub-account, recording the draw against the strategy's running `allocated` balance up to the limit its key resolves to.
- `transferFundBackToVault(amount)` returns `amount` of base asset from the strategy into the vault and reduces the strategy's `allocated` balance by the same amount.

Outbound funds therefore leave the vault via the vault's own `manage` hook, while inbound funds are pulled straight into the vault, so this manager never custodies the base asset in the normal path.

7 Findings

7.1 Low

7.1.1 [Low] Repayment path reverts if a blocklist-capable baseAsset blocks the strategy or the vault

File(s): `SubAccountFundManager.sol`

Description: `transferFundBackToVault(...)` repays only via `baseAsset.safeTransferFrom(msg.sender, address(boringVault), amount)`, with `baseAsset` and `boringVault` immutable and no alternative route.

Impact: If `baseAsset` is blocklist-capable (USDC is used in the aprimeUSD deployment scripts), a blocklist of the calling strategy or the `boringVault` recipient makes this transfer revert, so that strategy cannot repay and its `allocated` cannot be reduced. It needs the token issuer to blocklist a protocol-controlled address, is not attacker-triggerable, and freezes one strategy's repayment.

Recommendation(s): Provide a governance-controlled alternative repayment route, or accept and document the blocklist risk of the chosen `baseAsset`.

Status: Fixed

Update from Hyperwave: Added comment in PR-87

Update from CODESPECT: Fixed in commit `f4ff022`.

7.1.2 [Low] `transferFundFromVault(...)` uses unchecked `transfer(...)` call

File(s): `SubAccountFundManager.sol`

Description: `transferFundFromVault(...)` in `SubAccountFundManager.sol` moves the vault's `baseAsset` to the calling strategy by encoding a raw `transfer(...)` call:

```
boringVault.manage(
    address(baseAsset),
    abi.encodeWithSignature(
        "transfer(address,uint256)",
        msg.sender,
        amount
    ),
    0
);
```

The transfer call's result is never checked. A token that returns false instead of reverting on failure would cause this call to succeed at the EVM level while no tokens actually move. Meanwhile, `allocated[msg.sender]` has already been incremented before this call is made, so the strategy's allocation accounting would be updated even though the transfer silently failed.

Impact: Silent `transfer(...)` fails will cause accounting desync.

Recommendation(s): Check the `transfer(...)` call's successful execution.

Status: Fixed

Update from Hyperwave: Fixed in PR-87

Update from CODESPECT: Fixed in commit `972021b`.

7.1.3 [Low] `withdraw(...)` slippage floor misprices staker shares when `unstake` is true

File(s): `YieldBasisDecoderAndSanitizer.sol`

Description: The `withdraw(uint256 poolId, uint256 shares, uint256 minAssets, bool unstake, address receiver, bool withdrawStablecoins)` decoder derives its slippage floor from `preview_withdraw(shares)`, which interprets `shares` as LT shares. `unstake` is read but ignored and not bound in the merkle leaf, so a strategist can call `withdraw` with `unstake = true`, where `shares` is denominated in staker (gauge) shares instead of LT shares.

Impact: The configured staker (`0xd829456FD63Ada7DE0657714A3A7A26DE403E3D8`) is a non-1:1 ERC4626 vault over the LT (`convertToAssets(1e18) ≈ 0.985` LT, drifting as rewards accrue). With `unstake = true` the decoder prices staker shares as LT shares, overstating the withdrawal, so `minAllowed` is too high and the strategist must set `minAssets` above what the smaller actual LT withdrawal can deliver, reverting the call. No funds are at risk since the LT still enforces the strategist's `minAssets`; this blocks the staked-withdrawal path through the decoder and requires the position to have been staked first.

Recommendation(s): Bind `stake` / `unstake` in the merkle leaf so the decoder only authorizes the unstaked domain its floor assumes; if staked withdrawals are required, convert staker shares to LT shares via the staker's `convertToAssets(...)` before calling `preview_withdraw(...)`.

Status: Fixed

Update from Hyperwave: fixed in PR-87

Update from CODESPECT: Fixed in commit `bcfacae`.

7.2 Info

7.2.1 [Info] `transferFundBackToVault(...)` floors `allocated` to zero while transferring the full amount

File(s): `SubAccountFundManager.sol`

Description: `transferFundBackToVault(amount)` sets `allocated[msg.sender]` to `0` when `amount > allocated[msg.sender]`, but still calls `baseAsset.safeTransferFrom(msg.sender, address(boringVault), amount)` for the full `amount`. `allocated[msg.sender]` is only increased in `transferFundFromVault` when tokens leave the vault, so it never exceeds the strategy's actual debt and cannot be driven below it.

Impact: A strategy that returns more than it owes transfers the excess (`amount - allocated[msg.sender]`) into the vault while `allocated` is set to `0`, leaving the excess untracked.

Recommendation(s): Cap the decrease and the transfer to `allocated[msg.sender]`, or revert when `amount > allocated[msg.sender]`.

Status: Acknowledged

Update from Hyperwave: It is intended. After pulling fund from the vault and running strategy, sub account gets yield and repay amount can be greater than pulling amount

Update from CODESPECT: Acknowledged.

7.2.2 [Info] Slippage floors collapse to zero on `deposit(...)` and `withdraw(...)`

File(s): `YieldBasisDecoderAndSanitizer.sol`

Description: `deposit(...)` and `withdraw(...)` compute `minAllowed = (expected * (slippageBase - maxSlippage)) / slippageBase` (with `expected = preview_deposit(assets, debt, false)` / `preview_withdraw(shares)`) and revert only when `provided < minAllowed`, with no zero guard. When `expected` is small enough that `minAllowed` truncates to `0`, a `provided` of `0` passes, removing the only on-chain slippage bound, since these arguments are not in the merkle leaf.

Impact: When the floor is `0`, a `deposit(...)` / `withdraw(...)` passes with `minShares` / `minAssets = 0`, so the call runs with no on-chain slippage protection and accepts any output amount, exposing it to value loss from price movement or sandwiching. In practice the floor reaches `0` only for dust-level inputs (the previews return `0` or revert for sufficiently small `assets` / `shares`), so a meaningful deposit or withdraw is not affected on the current pool. The gap matters for other LTs or pool states where the previews can return `0` over a wider range, which is relevant since multiple pools are planned.

Recommendation(s): Revert when `expectedShares` / `expectedAssets` is `0`, and when the provided `minShares` / `minAssets` is `0`.

Status: Fixed

Update from Hyperwave: fixed in PR-87

Update from CODESPECT: Fixed in commit `7bdc0d9`.

7.2.3 [Info] `transferFundFromVault(...)` reverts for a registered strategy whose limit is unset or non-numeric

File(s): `SubAccountFundManager.sol`, `Configuration.sol`

Description: `transferFundFromVault(...)` reads the limit as a string with `configuration.getConfiguration(limitKey)` and parses it with `_parseUint`. `getConfiguration(...)` returns an empty string for an unset key, and `_parseUint` reverts with `ParsingFailed()` on an empty or non-numeric string.

Impact: A strategy registered through `setStrategyLimitKey(...)` whose `Configuration` value is never set (or set non-numeric) reverts on every `transferFundFromVault` until governance sets a valid number. It only blocks pulls, never allows an over-pull, and is fixed by setting a valid value.

Recommendation(s): Store the limit as a `uint256` per strategy, or validate the resolved value is a non-empty numeric string when the key is set.

Status: Acknowledged

Update from Hyperwave: The configuration contract is designed for storing all config values of the vault. It's general, so the value's type is string. The responsibility of setting the correct value in the configuration contract is of pdao (offchain). We have to set it via Safe, and signers have to verify the data

Update from CODESPECT: Acknowledged.

7.2.4 [Info] Merkle leaf binds decoder and target addresses but not their code

File(s): `SubAccountWithMerkleVerification.sol`

Description: `_verifyManageProof(...)` builds the leaf from the decoder and target addresses, the value flag, the selector, and the decoded address args, but not from their `codehash`, so the proof never covers the code deployed at those addresses.

Impact: If the code behind a whitelisted decoder or target address changes, a previously generated proof still verifies, so governance must re-issue `manageRoot` whenever a whitelisted integration changes. The path is pdao-only, the in-scope decoders are immutable, and any harm requires a third-party target to upgrade to hostile behaviour.

Recommendation(s): Bind `target.codehash` (and `decoder.codehash`) into the leaf, or whitelist immutable protocol-controlled adapters instead of raw upgradeable endpoints.

Status: Acknowledged

Update from Hyperwave: This is a deliberate design. Typically, the `argumentAddresses` arguments encoded in the decoder will not be changed unless the target contract function is changed. In case it changed, manageRoot will be updated via pdao.

Update from CODESPECT: Acknowledged.

7.2.5 [Info] Authorized wrap/unwrap leaves cannot execute against the non-payable sub-account

File(s): `SubAccountWithMerkleVerification.sol`, `CreateAprimeUSDYieldBasisMerkleRoot.s.sol`

Description: The merkle root authorizes wrap (`WETH.deposit(...)`) and unwrap (`WETH.withdraw(...)`) leaves, but `SubAccountWithMerkleVerification` has no `receive(...)`/ `fallback(...)` and a non-payable constructor, so it cannot hold or receive native ETH.

Impact: The unwrap reverts because `WETH.withdraw(...)` sends native ETH to the sub-account, and the wrap cannot be funded because the sub-account holds no native ETH. Both leaves are non-functional. No value is lost or frozen: WETH stays usable as an ERC20 through the `approve(...)`/ `deposit(...)`/ `withdraw(...)` leaves.

Recommendation(s): Remove the wrap/unwrap leaves from the root, or add `receive() external payable` if native-ETH flows are intended.

Status: Fixed

Update from Hyperwave: Fixed in commit PR-86

Update from CODESPECT: Fixed in commit `86764e1`.

7.2.6 [Info] Merkle authorization assumes the decoder and target share the same function for a given selector

File(s): `SubAccountWithMerkleVerification.sol`, `CreateAprimeUSDYieldBasisMerkleRoot.s.sol`

Description: `_verifyCallData(...)` builds the leaf from `bytes4(targetData)` and the decoder-returned addresses, then `manage(...)` sends the same `targetData` to `target`, so both dispatch on the same selector. The binding is only complete when the decoder function and the target function for that selector share the same signature and the decoder returns all of that function's address parameters. Nothing on-chain enforces this.

Impact: In the deployed roots the binding holds: the decoder functions are exact-signature mirrors of the bound targets, with `spender` / `receiver` / `newOwner` bound. A future leaf for a `(target, selector)` whose target function differs in signature from the decoder function (a cross-signature selector collision, 1/2^32) could bind the wrong address or none. It is not reachable today and depends on a governance-authored leaf.

Recommendation(s): In the merkle tooling, assert each leaf's signature resolves to the same selector on both decoder and target, and that the decoder returns every address parameter of the target function.

Status: Acknowledged

Update from Hyperwave: We implement the decoder and sanitizer and always have to ensure that the decoder and sanitizer is compatible with target contract.

Update from CODESPECT: Acknowledged.

7.2.7 [Info] `maxSlippage` and `slippageBase` are global, not configurable per `poolId`

File(s): `YieldBasisDecoderAndSanitizer.sol`

Description: `isAllowedPoolId` and `poolIdToLT` are configured per `poolId`, but `maxSlippage` and `slippageBase` are single values shared by the whole decoder. The team confirmed support for multiple pools is planned, so one slippage band would govern every pool the decoder serves.

Impact: With multiple pools enabled, the same slippage band applies to every pool. This band is a secondary check; the strategist-provided `min_shares` and `min_assets`, enforced by the LT, are the primary protection.

Recommendation(s): Configure `maxSlippage` and `slippageBase` per `poolId`.

Status: Fixed

Update from Hyperwave: Fixed in PR-87

Update from CODESPECT: Fixed in commit `9e24f4d`.

7.2.8 [Info] `withdraw(...)` reuses the deposit pool allowlist, so disabling a pool freezes existing positions

File(s): `YieldBasisDecoderAndSanitizer.sol`

Description: `deposit(...)` and `withdraw(...)` both check the same `isAllowedPoolId[poolId]` flag. `setAllowedPoolId(poolId, false)`, which governance uses to stop new deposits, also makes the `withdraw(...)` decoder revert for positions already in that pool.

Impact: Disabling a pool blocks the standard exit for positions already in it, and the revert aborts the whole `manageWithMerkleVerification(...)` batch. No funds are lost: re-enabling the pool restores the exit, and the pdao-only `manage` path can unwind the position without the decoder. It is reachable only through a `GOVERNANCE_ROLE` write.

Recommendation(s): Track entry and exit allowlists separately, or document that disabling a pool also blocks withdrawals from it.

Status: Fixed

Update from Hyperwave: Fixed in this PR PR-87

Update from CODESPECT: Fixed in commit `82337c5`.

7.2.9 [Info] Interleaved verify/execute makes the YieldBasis slippage floor depend on a live preview

File(s): SubAccountWithMerkleVerification.sol, YieldBasisDecoderAndSanitizer.sol

Description: manageWithMerkleVerification(...) verifies and executes each call in the same loop iteration: _verifyCallData(i) is followed immediately by manage(i), so call i is verified only after calls 0..i-1 have already changed state. For deposit and withdraw, the decoder derives the slippage floor from the live ILT.preview_deposit(...) / preview_withdraw(...) value (deposit_crvusd and redeem_crvusd include no floor).

Impact: An earlier call in the same batch can move the state a later call's preview reads, so the decoder's slippage floor for that later call is computed against already-mutated state. The strategist-provided min_shares / min_assets still cap the loss, but if lt is spot-priced (set via setPoolLT(...)), the decoder's floor can be manipulated.

Recommendation(s): Verify all calls in the batch before executing any of them, or document that the decoder floor is a secondary check dependent on live preview state.

Status: Fixed

Update from Hyperwave: fixed in PR-87

Update from CODESPECT: Fixed in commit 91e80ac.

7.2.10 [Info] Direct manage(...) call will execute successfully even when the contract is paused

File(s): SubAccountWithMerkleVerification.sol

Description: manage(...) in SubAccountWithMerkleVerification.sol is declared public requiresAuth, meaning it is independently callable as an external entry point and not restricted to being invoked only internally by manageWithMerkleVerification(...). The isPaused check is only enforced in the manageWithMerkleVerification(...), and since manage(...) itself contains no isPaused check, it can be called while contract is paused.

Impact: The pause() mechanism can be silently bypassed.

Recommendation(s): Add the isPaused check directly inside manage(...) function.

Status: Fixed

Update from Hyperwave: fixed in PR-87

Update from CODESPECT: Fixed in commit fee2ad4.

7.2.11 [Info] Missing zero-address validation in SubAccountFundManager constructor

File(s): SubAccountFundManager.sol

Description: The constructor in SubAccountFundManager assigns all three immutables directly from constructor arguments with no address(0) validation. In contrast, YieldBasisDecoderAndSanitizer does check the passed argument for address(0) before usage.

Impact: Since these variables are immutable, a deployment-time mistake is unrecoverable without redeploying the contract.

Recommendation(s): Add zero-address checks for constructor arguments.

Status: Fixed

Update from Hyperwave: fixed in PR-87

Update from CODESPECT: Fixed in commit 7830842.

8 Threat Model

Methodology

This threat model uses **actor-capability analysis** as its spine, cross-checked against an **invariant-violation matrix**. The actor lens is the right primary framing because the in-scope contracts are permissioning and accounting layers between the vault's funds and external strategies: their risk is dominated by what the parties they trust (the `Auth` owner, the decoder governance, and the semi-trusted strategists and strategy sub-accounts) can do within, or by escaping, the sandboxes the framework builds around them. We therefore enumerate, for each actor, what it may do when honest, its blast radius if compromised or mistaken, and what bounds it, whether a Merkle allow-list, a decoder check, an allocation limit, or the `Auth` gate.

The invariant matrix is the rigour layer: each property the framework is expected to preserve (a strategist can only make pre-committed calls, address arguments are bound by the tree, a sub-account cannot exceed its allocation limit, and slippage on strategy calls is bounded) is phrased as an attacker goal and matched against its mitigation. Where useful, threats are tagged with the informal STRIDE categories *Spoofing*, *Tampering*, *Elevation of privilege*, and *Denial of service*.

Out of Scope

The vault (`BoringVault`) and its `manage` execution hook, the `Auth`/`Authority` access-control layer, the `Configuration` store, and the `BaseDecoderAndSanitizer` base contract are out of scope as software; the review treats them as trusted and models only the in-scope contracts' interaction with them. The external protocols the strategies call (the Yield Basis leveraged token and its previews, crvUSD, and the wrapped native token) are likewise out of scope, but the *consequences* of trusting their return values and of governance misconfiguring the decoder are modelled below. The correctness of the off-chain construction of each strategist's Merkle tree is an operational assumption; the on-chain effect of the tree that is actually set is modelled.

Assets

The following are what an attacker would seek to divert, corrupt, or deny, ranked roughly by impact:

a. **The vault's base asset.** The funds the framework allocates to strategies. The whole point of the two sandboxes is that a strategist can only move and deploy these funds in pre-approved ways and only up to a configured limit.

b. **The integrity of the call allow-list.** That a strategist can execute only calls committed in their Merkle root, against the committed target and decoder, with only the committed address arguments. Any escape from this allow-list widens what a semi-trusted strategist can do with vault funds.

c. **Allocation-limit correctness.** That `allocated[strategy]` never exceeds the strategy's `Configuration`-resolved limit, and that the accounting faithfully tracks outstanding funds.

d. **Slippage and target-protocol bounds.** That deposits and withdrawals into the Yield Basis strategy respect the governance-configured slippage floor, so a strategist cannot route value out through an intentionally bad min-out.

e. **Availability.** That the router can be paused, and the fund manager constrained by the owner (via `Auth` and limit-key revocation), in an emergency without stranding funds.

Trust Boundaries

- **Router ↔ strategist (dominant boundary).** The per-strategist `manageRoot` plus the decoder is the boundary control. The Merkle leaf binds the decoder, target, value-non-zero flag, selector, and address arguments of every allowed call; the decoder staticcall is what turns the raw calldata into the packed addresses the leaf commits, and is also free to revert on input the tree cannot police.
- **Router ↔ decoder.** The router trusts each decoder named in a leaf to extract addresses faithfully and to sanitize. A wrong or over-permissive decoder loosens the sandbox even if the tree is correct; the decoder is therefore a trusted component, and for `YieldBasisDecoderAndSanitizer` its governance is trusted to set sane pools, leveraged tokens, and slippage.
- **Fund manager ↔ sub-account.** A strategy is recognised only by a non-empty `strategyLimitKey`, and can pull funds only up to the limit that key resolves to through `Configuration`. Recognition (the key) and authorisation (`Auth`) are two independent gates.
- **Contracts ↔ vault (final custody).** Outbound transfers go through `boringVault.manage`; the vault is the custodian and the final authority on whether a transfer executes, so the fund manager must itself be an authorised vault manager.
- **Owner ↔ everything (fully trusted).** The `Auth` owner/authority sets roots and limit keys, pauses, and may call the standalone `manage` with no Merkle constraint; it is outside the sandbox by design.

Threat Actors

Each actor is described by its honest capabilities, its blast radius if compromised or mistaken, and the bounds that constrain it.

Strategist (drives the router)

Honest capabilities: submit batches of calls to `manageWithMerkleVerification`, each with a Merkle proof against their own root, to deploy allocated funds into approved strategies.

If the strategist is hostile or its key is compromised: the blast radius is exactly the set of call shapes in that strategist's Merkle tree, further narrowed by the decoders' runtime checks. The attacker cannot call an un-committed target, use an un-committed decoder, pass an un-committed address argument, or, for the Yield Basis strategy, deposit or withdraw with a min-out below the governance-configured slippage floor. It cannot reach vault funds beyond what the fund manager's limit permits. The attacker is confined to the intersection of the tree and the decoder policy. *(Elevation of privilege / Tampering.)*

Strategy sub-account (pulls and returns funds)

Model: an address registered with a non-empty `strategyLimitKey` that calls `transferFundFromVault` and `transferFundBackToVault`.

Threats and bounds: a sub-account can pull base asset only while `allocated + amount ≤ limit`, so its outstanding draw is capped by the `Configuration` value its key resolves to; an unregistered (empty-key) caller fails closed with `InvalidSubAccount`, and a registered key whose `Configuration` value is unset or non-numeric fails closed with `ParsingFailed`. On return, `allocated` is decremented and floored at zero, so it never underflows. *(Tampering.)*

Decoder governance (Yield Basis)

Model: the holder of `GOVERNANCE_ROLE` on `YieldBasisDecoderAndSanitizer`, a self-administered role, configures the pool allow-list, the per-pool leveraged-token address, and the slippage bounds.

Threats and bounds: governance is trusted, but its settings directly define the decoder's policy: a wrong `poolIdToLT`, an over-permissive slippage, or an allow-listed pool that should not be reachable weakens the on-chain checks applied to every proven deposit and withdraw. The `_validateSlippageSettings` guard keeps the slippage fraction well-formed (`maxSlippage < slippageBase`, non-zero base), but does not judge whether the chosen tolerance is economically safe. *(Tampering / Elevation of privilege.)*

Configuration store (limit source)

Model: the external `Configuration` contract returns the decimal-string limit for each strategy key.

Threats and bounds: the fund manager trusts this value; an empty or non-numeric string makes `_parseUint` revert `ParsingFailed` (failing the transfer closed rather than defaulting to an unbounded limit), while a raised limit widens a sub-account's draw. The store is trusted infrastructure, and its correct administration is an operational assumption. *(Tampering.)*

Owner / Authority (privileged)

Model: the `Auth` owner, or an `Authority` it installs, holds every `requiresAuth` power on both contracts.

Threats and bounds: the owner sets and revokes strategist roots and strategy limit keys, pauses the router, and may call the standalone `manage` directly, which bypasses the Merkle proof, the decoder staticcall, and the pause flag. This is a deliberate emergency escape hatch and places the owner fully inside the trusted boundary; its compromise is out of the framework's control and bounded only by the vault's own authorisation of these contracts. *(Elevation of privilege.)*

External strategy protocols

Model: the Yield Basis leveraged token (queried through `preview_deposit`/`preview_withdraw`), crvUSD, and the wrapped native token that strategist calls ultimately reach.

Threats and bounds: the slippage floor is measured against the leveraged token's own preview quote, so the check is only as trustworthy as that quote; the framework treats these protocols as trusted and does not model manipulation of their internal accounting, which is out of scope. *(Tampering.)*

Invariant / Violation Matrix

Invariant	How it could be violated	Mitigation
A strategist can only execute calls committed in their own Merkle root.	Proof forged, wrong root loaded, or a call shape not in the tree accepted.	Per-strategist `manageRoot` keyed by `msg.sender`; leaf rebuilt from decoder output, target, value flag, and selector; `MerkleProofLib.verify` reverts on mismatch.
Every allowed call's address arguments are bound by the tree.	Decoder returns wrong or omitted addresses; non-committed spender/receiver slips through.	Router `staticCall`s the leaf's decoder and folds its packed addresses into the leaf; a revert or mismatch aborts the batch.
Only implemented, sanitised strategy calls execute.	An unimplemented selector or out-of-policy input is forwarded.	Decoder `fallback` reverts `FunctionNotImplemented`; `YieldBasisDecoderAndSanitizer` rejects non-allowlisted pools, unset leveraged tokens, and min-out below the slippage floor.
A sub-account never draws more than its allocation limit.	Cumulative pulls exceed the limit, or an unregistered address pulls funds.	`transferFundFromVault` requires a non-empty `strategyLimitKey` and `allocated + amount` $\leq$ the `Configuration`-resolved limit; parse failure reverts closed.
Allocation accounting cannot underflow.	Returning more than was allocated corrupts the running total.	`transferFundBackToVault` floors `allocated` at zero on over-return.
Deposits and withdrawals respect the configured slippage.	Strategist supplies a deliberately loose min-out to bleed value.	Decoder enforces `minShares`/`minAssets` $\geq$ a slippage-bounded fraction of the leveraged token's preview quote.
The router can be halted in an emergency.	Strategist activity continues during an incident.	`pause`/`unpause` gate `manageWithMerkleVerification`; the owner retains the direct `manage` escape hatch.

Table 4: Invariant / violation matrix for the sub-account strategy framework.

9 Evaluation of Provided Documentation

The in-scope change set was documented in two complementary forms: the pull request #86 description, which fixes the scope and the reviewed commit, and in-code NatSpec docstrings on the reviewed contracts.

- **Pull request description:** The PR enumerates the contracts under review and pins the audited commit. It frames the change as the introduction of the sub-account strategy framework on top of the existing `BoringVault` codebase, which sets the boundary between the reviewed contracts and the surrounding vault, access-control, and configuration machinery that is treated as trusted.
- **In-code NatSpec:** Documentation is applied unevenly but is strongest where it matters most. `SubAccountFundManager` carries a full contract-level `@title`/`@notice`/`@dev` header and per-function `@notice`, `@dev`, and `@param` tags that spell out the allocation-limit model, the `limitKey` resolution against the `Configuration` store, and the direction of each transfer. `SubAccountWithMerkleVerification` documents the leaf composition explicitly, stating that each leaf is `keccak256(abi.encodePacked(decodersAndSanitizer, target, valueIsNonZero, selector, argumentAddress_0, ..., argumentAddress_N))` and what each component means, which is the single most important thing to get right when reading the manager. The two decoder-and-sanitizer contracts are more lightly commented: `YieldBasisDecoderAndSanitizer` annotates its slippage-bounding rationale inline, while `WrappedNativeDecoderAndSanitizer` is self-evident from its two one-line functions.

The documentation is adequate for a change set of this size and is clearest on the security-critical pieces (the Merkle leaf layout and the allocation-limit flow). Because the decoder-and-sanitizer contracts encode which external calls a strategist may make, **CODESPECT** recommends maintaining an explicit, per-strategy inventory of the target functions each decoder covers and the address arguments it commits, so that the Merkle allow-list stays legible as strategies are added. A detailed documentation assessment will be completed during the engagement.

10 Test Suite Evaluation

10.1 Compilation output

```
> forge compile
[] Compiling...
[] Compiling 145 files with Solc 0.8.23
[] Compiling 473 files with Solc 0.8.21
[] Solc 0.8.23 finished in 6.85s
[] Solc 0.8.21 finished in 194.70s
Compiler run successful with warnings: <removed for the size and out-of-scope files warnings>
```

10.2 Tests Output

```
> forge test --match-path 'test/strategist/aprimeusd/YieldBasis*.t.sol' --skip 'test/strategist/hwhype/*'

Ran 16 tests for test/strategist/aprimeusd/YieldBasisSubAccount.t.sol:YieldBasisSubAccountTest
[PASS] testFundManagerUnauthorizedCaller() (gas: 45229)
[PASS] testNonStrategistCannotManage() (gas: 127350)
[PASS] testRevertFailedToVerifyManageProof() (gas: 143108)
[PASS] testRevertInvalidDecodersAndSanitizersLength() (gas: 29007)
[PASS] testRevertInvalidManageProofLength() (gas: 28881)
[PASS] testRevertInvalidSubAccount() (gas: 332063)
[PASS] testRevertInvalidTargetDataLength() (gas: 28800)
[PASS] testRevertInvalidValuesLength() (gas: 29050)
[PASS] testRevertLimitExceeded() (gas: 344815)
[PASS] testRevertWhenPaused() (gas: 324043)
[PASS] testRoundTrip() (gas: 483144)
[PASS] testSetConfigurationUnauthorized() (gas: 16943)
[PASS] testSetManageRootUnauthorized() (gas: 28498)
[PASS] testSetStrategyLimitKeyUnauthorized() (gas: 28982)
[PASS] testTransferFundBackToVaultSuccessfully() (gas: 486803)
[PASS] testTransferFundFromVaultSuccessfully() (gas: 391054)
Suite result: ok. 16 passed; 0 failed; 0 skipped; finished in 1.04s (1.29s CPU time)
Ran 13 tests for test/strategist/aprimeusd/YieldBasis.t.sol:YieldBasisTest
[PASS] testDepositCrvusd() (gas: 423206)
[PASS] testDepositWeth() (gas: 3524850)
[PASS] testHybridVaultCreated() (gas: 13698)
[PASS] testRedeemCrvusd() (gas: 603830)
[PASS] testRevertApproveWrongSpender() (gas: 139333)
[PASS] testRevertDepositMinSharesTooLow() (gas: 1532249)
[PASS] testRevertDepositSlippage() (gas: 2217705)
[PASS] testRevertDisallowedPoolId() (gas: 290384)
[PASS] testRevertNonStrategist() (gas: 279738)
[PASS] testRevertWhenPaused() (gas: 307128)
[PASS] testRevertWithdrawMinAssetsTooLow() (gas: 3770360)
[PASS] testRevertWithdrawWrongReceiver() (gas: 3776434)
[PASS] testWithdrawWeth() (gas: 3948452)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 5.62s (27.29s CPU time)
Ran 2 test suites in 5.63s (6.65s CPU time): 29 tests passed, 0 failed, 0 skipped (29 total tests)
```

10.3 Notes on the Test Suite

All 29 in-scope tests pass across both fork suites, and coverage is solid overall. The core risk surface (deposit/withdraw/redeem flows, pool allowlist, slippage bounds, access control, pausing, and merkle-proof validation) is well tested with both happy-path and revert cases. The main gap was the untested unstake withdraw branch, but nothing here blocks the audit. Overall, the coverage is adequate for the scale of changes and maintains the proper testing framework used across the project.